

CS 32 - Problem Set 6

Diogn Lei R. Mortera

Question 1

You are given an $r \times c$ grid. Each cell of the grid is either empty or contains an obstacle.

It is possible to move from an empty cell to an adjacent empty cell. However, it is not allowed to move to a cell containing an obstacle, or exit the grid.

How many “rooms” are there in the grid?

A room is a *maximal* set of cells that you can go from any cell to any other cell using only the movement rules above.

You may assume that single-source DFS is correct, i.e., DFS from a cell (i, j) correctly visits precisely the cells reachable from (i, j) .

Problem 1.1: Find an $O(rc)$ algorithm that solves this problem by calling DFS multiple times.

```
func number_of_rooms(grid):
    rows, cols := number of rows and columns in the grid
    done = [rows x cols array, initially false]
    ans = 0

    func neighbors(loc):
        r, c = loc
        for (dr, dc) in [(1,0),(0,1),(-1,0),(0,-1)]:
            nr, nc = r+dr, c+dc

            if 0 < nr <= rows and
                0 < nc <= cols and
                grid[nr][nc] != '#' and
                and !done[nr][nc]:
                    yield (r+nr, c+nc)

    func dfs(loc):
        for (_i, _j) in neighbors(loc):
            done[_i][_j] = true
            dfs(_i, _j)

    for i from 0 to n-1:
        for j from 0 to n-1:
            if done[i][j] or grid[i][j] = '#': continue
            else:
                done[i][j] = true
                k := k + 1
                dfs(i,j)

    return k
```

Solution (Variant 2): Let the following algorithm above be `number_of_rooms`.

`number_of_rooms` initializes a `in_room` array to keep track of which cells already belong in a room. Then, for each cell (i, j) in the grid:

- If this cell belongs in the room or is blocked, it is skipped.
- If the cell does not belong to a room, then `dfs( $i, j$ )` is called to visit all of the rooms reachable from this cell to create a new room. Because `dfs( $i, j$ )` visits all cells reachable from (i, j) , so all cells marked done from this call belongs in the same room. To account for the new room, `ans` is incremented.

Let r and c be the number of rows and columns in the grid, respectively. Allocating the `in_room` array of $r \times c$ cells already takes exactly $r \cdot c$ operations in the array, implying $\text{number_of_rooms} = \Theta(rc)$.

Also, because `done` ensures a cell is not processed by `dfs` more than once, the total work across all `dfs` calls is at most rc (some cells can be blocked with `#`). This implies $\text{number_of_rooms} = \mathcal{O}(rc)$. The other operations in `number_of_rooms` run in $\Theta(1)$ time.

Hence, $\text{number_of_rooms} = c \cdot \Theta(1) + \Theta(rc) + \mathcal{O}(rc) = \mathcal{O}(rc)$ (although $\Theta(rc) + \mathcal{O}(rc) = \Theta(rc)$ also, which is a stronger result).

■

Solution (Old Variant 1): Let the following algorithm above be `number_of_rooms`.

The algorithm above initializes a `in_room` array to keep track of which cells have been visited. Then, for each cell (i, j) in the grid, it checks if it has been visited or not.

- If it has not been visited, then the cell belongs to a new room. k is incremented and `dfs( $i, j$ )` is called to visit all of the rooms reachable from this cell, also marking them in `in_room` so they will not be revisited.
- If a cell has been visited, then the cell already belongs to a room that has been accounted for and is therefore skipped.

Let r and c be the number of rows and columns in the grid, respectively. Allocating the `in_room` array of $r \times c$ cells already takes exactly $r \cdot c$ operations in the array, implying $\text{number_of_operations} = \Theta(rc)$.

Also, because `in_room` ensures a cell is not processed by `dfs` more than once, the total work across all `dfs` calls is at most rc (some cells can be blocked with `#`). This implies $\text{number_of_operations} = \mathcal{O}(rc)$. The other operations in `number_of_operations` run in $\Theta(1)$ time.

Hence, $\text{number_of_operations} = \Theta(rc) + \mathcal{O}(rc) = \Theta(rc)$.

■

Problem 1.2: Formally prove your solution is correct.

Proof: Let r and c be the number of rows and columns in the grid, respectively. Suppose the cells were visited in increasing i , then increasing j . If $t := i \cdot c + j$, we use the following invariant applying to `number_of_rooms` for $0 \leq i < r$ and $0 \leq j < c$:

Right before iteration t of the loop, if there are k rooms that intersect with the first t cells, then `ans` contains k .

Initialization.

At $t = 0$, no cells have been visited yet. Hence, `ans` = 0.

Maintenance.

Suppose before iteration t , the invariant holds. That is, `ans` contains k if k rooms that intersect the first t cells.

During iteration t , some cell (i, j) distinct from the other cells is called in the inner loop. There are three cases:

- `grid[i][j] = '#'`. Then this cell is blocked and cannot be in a room. Hence, k stays the same and `ans` stays the same.
- `in_room[i][j] = true`. Because `in_room[i][j] := true` before iteration t , it is not the origin `dfs` call that set it to true. So some other cell (i', j') processed at some $t' < t$ found `in_room[i'][j'] := false`, triggering `dfs(i', j')` which marked all cells reachable from (i', j') (e.g. `in_room[i][j] = true`), including (i, j) . Hence, by definition, (i, j) and (i', j') belong in the same room. Thus, by the inductive hypothesis, this room is already one of the rooms counted in the inductive hypothesis. Therefore, this iteration leaves `ans` unchanged, hence the invariant is maintained.
- `in_room[i][j] = false`. Then this cell must not intersect with any of the k rooms. Hence, it does not belong in a room. A new room is marked by calling `dfs(i, j)`, correctly marking all cells reachable from (i, j) as done in the `in_room` array (to mark them as belonging already in a room) and incrementing `ans` correspondingly to $k + 1$ rooms.

In either case, the invariant holds right after the iteration.

Termination.

After all cells have been visited, the first rc cells (or the whole grid) are covered. Hence, `ans` contains the right answer for the number of rooms in the grid.

■

Question 2

You are given an $n \times n$, where each cell is either empty or contains an obstacle, and a cell (i, j) , $0 \leq i, j < n$.

Suppose that the following are performed:

- First, DFS is performed once starting from (i, j) , computing the parent information along the way.
- Next, for each reachable cell (i', j') , a path from (i, j) to (i', j') is reconstructed as a list of pairs.
- Finally, the list of all these paths is returned.

For which $k \in \{1, 2, 3, 4, 5\}$ is the following statement true? (And prove it!)

(can we assume neighbors is edgely-adjacent?)

Claim 1: The values that work are

$$k = \{3, 4, 5\}$$

Proof:

Let our the running time of the algorithm above be $f(n)$:

DFS takes at most $O(n^2)$ time. Meanwhile, by Claim 1, enumerating the lists themselves take $\Theta(n^3)$ time. Therefore, the algorithm above runs in:

$$O(n^2) + \Theta(n^3) = \Theta(n^3)$$

which implies $f(n) = \Omega(n^3)$ and $f(n) = O(n^3)$.

Hence, $f(n) = O(n^3) = O(n^k)$ for $k = 3, 4, 5$ work.

Next, we also want to show that $f(n) \neq O(n^k)$ with $k < 3$. We will prove this using contradiction, assuming $f(n) = O(n^k)$.

By definition of $\Omega(n^3)$, there must exist a c_1 and n_1 such that for all $n \geq n_1$:

$$c_1 n^3 \leq f(n)$$

Suppose $f(n) = O(n^k)$ with $k < 3$ for this algorithm. Then there must exist a c_2 and n_2 such that for all $n \geq n_2$:

$$f(n) \leq c_2 n^k$$

We can chain the inequalities:

$$\begin{aligned} c_1 n^3 &\leq c_2 n^k \\ n^3 &\leq \frac{c_2}{c_1} n^k \end{aligned}$$

This implies $n^3 = O(n^k)$ (with constants $\frac{c_2}{c_1}, \max(n_1, n_2)$), but $k < 3$, a contradiction. Hence, $f(n) \neq O(n^k)$.

Hence, the only valid k are those ≥ 3 $\{3, 4, 5\}$. ■

Claim 2: The enumeration of the paths themselves run in $\Theta(n^3)$ time.

Proof: Because DFS returns the shortest path to any cell, a cell that is d Manhattan distances away has a path length of d .

Note that there are c cells that has a distance of c (if $c \leq \lfloor \frac{n}{2} \rfloor$), and $n - c$ cells otherwise.

Therefore, the sum of cells within the list of all paths can be expressed as:

$$\sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} i \cdot i + \sum_{i=\lceil \frac{n}{2} \rceil}^n i(n-i)$$

The LHS can be bounded by $\frac{1}{48}$:

$$\begin{aligned} \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} i \cdot i + \sum_{i=\lceil \frac{n}{2} \rceil}^n i(n-i) &\geq \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} i \cdot i \\ &= \frac{\lfloor \frac{n}{2} \rfloor (\lfloor \frac{n}{2} \rfloor + 1) (2 \cdot \lfloor \frac{n}{2} \rfloor + 1)}{6} \\ &\geq \frac{n}{24} \cdot \frac{n}{24} \cdot \frac{n}{24} \quad \left(\frac{n}{24} \leq \frac{\frac{n}{2} - 1}{6} \leq \frac{\lfloor \frac{n}{2} \rfloor}{6} \text{ for } n \geq 4 \right) \\ &= \frac{n^3}{24^3} \end{aligned}$$

The RHS can be bounded by $\frac{1}{8}$:

$$\begin{aligned} \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} i \cdot i + \sum_{i=\lceil \frac{n}{2} \rceil}^n i(n-i) &\leq \sum_{i=0}^n i \cdot i \\ &= \frac{n(n+1)(2n+1)}{6} \\ &\leq \frac{n}{2} \cdot \frac{n}{2} \cdot \frac{n}{2} \quad \left(\frac{2n+1}{6} \leq \frac{n}{2} \text{ for } n \geq 1 \right) \\ &= \frac{n^3}{8} \end{aligned}$$

Hence, by definition, the enumeration of the paths already run in $\Theta(n^3)$ time (with constants $\frac{1}{48}$, $\frac{1}{8}$, and $\max(1, 4) = 4$).

■.

Problem 3

Here, we'd like to show that there are *many* paths in a grid, by providing an asymptotic lower bound. Prove that there are $\omega(b^{n^2})$ paths in an $n \times n$ grid from $(0, 0)$ to $(n - 1, n - 1)$ for some $b > 1$.